

实验报告 4 —— 图像压缩实验【严禁抄袭！】

by Kureha Shiro Dec.2025

一、实验目的

- (1) 了解降维算法和 PCA 的基本概念；
- (2) 了解 Auto-Encoder 模型的用途与搭建方法；
- (3) 了解无监督学习的相关知识。

二、实验原理

2.1 降维 (Dimensionality Reduction)

简单来说，**降维**就是减少数据集特征数量的过程。它通过某种数学变换，将原始高维空间中的数据点，映射到一个新的、维度更低的空间中。

降维绝非简单地丢弃数据，其核心价值在于：

(1) 可视化：人类最多能直观理解三维空间。通过降维到 2D 或 3D，我们可以将高维数据画出来，直观地观察其结构、分组和异常点。

(2) 提升效率：更少的数据维度意味着更小的存储空间、更快的训练速度和更低的计算成本。

(3) 去除噪声与冗余：许多算法（尤其是距离计算类算法如 KNN）在高维空间中会因不相关或重复的特征而性能下降。降维可以提炼出数据的精华。

(4) 缓解维度灾难：在高维空间中，数据会变得极其稀疏，导致许多机器学习模型难以找到有效的模式。

降维的核心思想：信息保留。关键挑战是：如何在降低维度的同时，最大限度地保留原始数据中有价值的信息（如方差、数据结构）？

2.2 主成分分析 (PCA)

PCA (Principal Component Analysis) 是一种使用最广泛的数据降维算法，常用于高维数据的降维，可用于提取数据的主要特征分量。其主要思想是**将 n 维特征映射到 k 维上**，这 k 维是全新的正交特征，也被称为主成分，是在原有 n 维特征的基础上重新构造出来的 k 维特征。其算法步骤如下：

1. 将原始数据按列组成 n 行 m 列矩阵 X ；
2. 将 X 的每一行进行零均值化，即减去这一行的均值；
3. 求出协方差矩阵 $C = \frac{1}{m} XX^T$ ；
4. 求出协方差矩阵的特征值及对应的特征向量；
5. 特征向量按对应特征值大小从上到下按行排列成矩阵，前 k 行组成矩阵 P ；
6. $Y = PX$ 即为降维到 k 维后的数据。

2.3 自编码器（Auto-Encoder）

自编码器（Auto-Encoder）是一种用于无监督学习的神经网络模型，它的基本原理可以概括为以下几个步骤：

1. **编码器（Encoder）**：这部分的网络将输入数据压缩成一个较低维度的表示形式（称为编码）。这个过程类似于数据压缩，目的是提取输入数据中最重要特征。

2. **编码（Code）**：编码是编码器处理后的结果，是输入数据的一个压缩表示。通常是一个远小于输入数据维度的向量，捕捉了输入数据的核心特征。

3. **解码器（Decoder）**：解码器部分尝试从编码中重构原始输入数据。这个过程可以看作是对编码的解压缩。理想情况下，解码器的输出应该与原始输入数据尽可能相似。

4. **损失函数（Loss Function）**：自编码器的训练过程中，需要一个损失函数来度量解码器输出和原始输入之间的差异。常用的损失函数包括均方误差（MSE）等。通过最小化这个损失，网络学习到如何有效地编码和解码数据。

三、实验过程及结果

3.1 网络结构定义错误修正

在调试原始代码过程中发现，Notebook 的[11]、[12]两格都定义了 CNN 图像识别网络（AutoEncoder_Net 类），程序能跑完不过是[11]被[12]覆盖了。

```
[11] # 定义CNN图像识别网络
class AutoEncoder_Net(nn.Cell):
    def __init__(self, in_channel, embeded_channel=2):
        super(AutoEncoder_Net, self).__init__()
        self.encoder = nn.SequentialCell([
            nn.Dense(in_channel, 400),
            nn.ReLU(),
            nn.Dropout(),
            nn.Dense(400, embeded_channel),
            nn.ReLU()
        ])
        self.decoder = nn.SequentialCell([
            nn.Dense(embeded_channel, 400),
            nn.ReLU(),
            nn.Dense(400, in_channel),
            nn.Sigmoid(),
        ])
        self.flatten = nn.Flatten()
    #构建模型
    def construct(self, x):
        x = self.flatten(x)
        lowData = self.encoder(x)
        reconData = self.decoder(lowData)
        return x, lowData, reconData

[12] # 定义CNN图像识别网络
class AutoEncoder_Net(nn.Cell):
    def __init__(self, in_channel, embeded_channel=2):
        super(AutoEncoder_Net, self).__init__()
        self.encoder = nn.SequentialCell([
            nn.Dense(in_channel, 512),
            nn.ReLU(),
            nn.Dropout(),
            nn.Dense(512, 128),
            nn.ReLU(),
            nn.Dropout(),
            nn.Dense(128, embeded_channel),
            nn.ReLU()
        ])
        self.decoder = nn.SequentialCell([
            nn.Dense(embeded_channel, 128),
            nn.ReLU(),
            nn.Dense(128, 512),
            nn.ReLU(),
            nn.Dropout(),
            nn.Dense(512, in_channel),
            nn.Sigmoid(),
        ])
        self.flatten = nn.Flatten()
    #构建模型
    def construct(self, x):
        x = self.flatten(x)
        lowData = self.encoder(x)
        reconData = self.decoder(lowData)
        return x, lowData, reconData
```

图 1 原代码对 CNN 图像识别网络的两种不同定义

两种网络结构的差异对比如下：

[11] 是一个较浅的 4 层网络（Encoder 2 层，Decoder 2 层）

结构：Input→400→30→400→Output；

参数量较少，拟合能力较弱。

[12] 是一个较深的 6 层网络 (Encoder 3 层, Decoder 3 层)

结构: Input→512→128→30→128→512→Output;

参数量更多, 且使用了“漏斗形”逐层降维的设计。模型更深、神经元更多 (512/128 vs 400)。对于图像压缩任务, 逐层提取特征 (512→128→30) 显然要比直接从输入压缩到 400 再到 30 效果更好。

3.2 原始程序运行结果

任务一 (5') : 实现 PCA 对图像的压缩, 粘贴得到的部分图像结果。

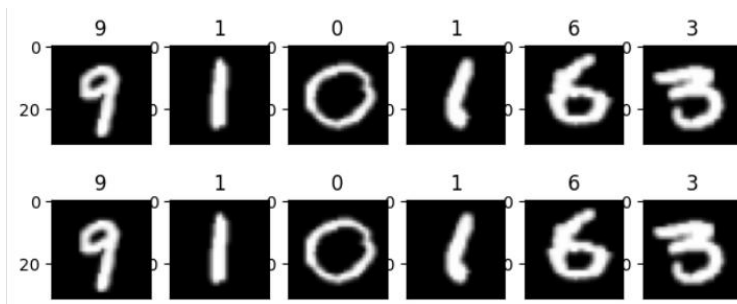


图 2 原始程序 PCA 压缩结果

任务二 (10') : 运行 baseline, 粘贴得到的部分图像结果, 比较分别使用 PCA 和 Auto-Encoder 压缩并复原后的图像效果, 进行一定的分析。

```
===== Starting Training =====
epoch: 1 step: 100, loss is 0.05475565046072006
epoch: 1 step: 200, loss is 0.05457134544849396
epoch: 1 step: 300, loss is 0.04999062418937683
epoch: 1 step: 400, loss is 0.04942106083035469
epoch: 2 step: 32, loss is 0.04759027808904648
epoch: 2 step: 132, loss is 0.04664064943790436
epoch: 2 step: 232, loss is 0.04479559510946274
epoch: 2 step: 332, loss is 0.0443100780248642
epoch: 2 step: 432, loss is 0.041142359375953674
epoch: 3 step: 64, loss is 0.03799033910036087
epoch: 3 step: 164, loss is 0.03201659023761749
epoch: 3 step: 264, loss is 0.031185153871774673
epoch: 3 step: 364, loss is 0.031047802418470383
epoch: 3 step: 464, loss is 0.029514417052268982
epoch: 4 step: 96, loss is 0.028894010931253433
epoch: 4 step: 196, loss is 0.028383666649460793
epoch: 4 step: 296, loss is 0.02770514227449894
epoch: 4 step: 396, loss is 0.028844846412539482
epoch: 5 step: 28, loss is 0.02697126567363739
epoch: 5 step: 128, loss is 0.024904651567339897
epoch: 5 step: 228, loss is 0.028025023639202118
epoch: 5 step: 328, loss is 0.027204548940062523
epoch: 5 step: 428, loss is 0.024757809937000275
===== finish Training =====
```

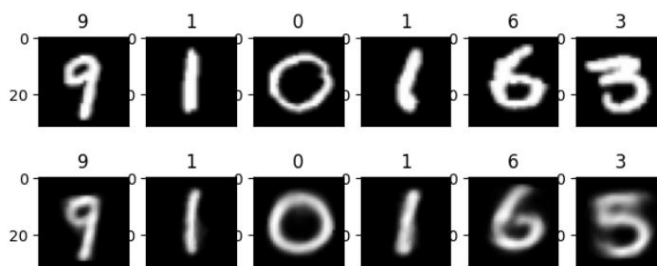


图 3 原始程序 Auto-Encoder 训练过程及压缩结果

看到图 3, 我们不禁要问: 为什么 Auto-Encoder 压缩并复原后的图像出现了畸变 (最后一个数字 3 变成了 5, 即便模型仍能够神秘地识别为 3, 但肉眼观察显然是 5)。大概是因为原始代码里的模型欠拟合。具体原因有三点:

- (1) 训练时间太短: 只训练了 5 个 Epoch, 只记住大概轮廓, 记不清细节。
- (2) 特征混淆: 数字“3”和“5”结构非常相似。在没训练好的情况下, 模型把这两个数字在潜空间里搞混了。
- (3) MSE 损失函数的特性: 当模型分不清是 3 还是 5 时, 为了让整体误差最小, 它会选择输出一个“3 和 5 的平均值”。这个模糊的“平均怪胎”在视觉上碰巧看起来更像 5。

改进方案：增加训练轮数 epochs 到 20.

代码段 1：修改训练轮数 epochs

```
model.train(20, train_ds, callbacks=[loss_cb, ckpoint_cb], dataset_sink_mode=False)
```

以下是两种模型压缩并复原后的图像效果直观视觉对比以及特性总结。

表 1 使用 PCA 和 Auto-Encoder 压缩并复原后的图像效果直观视觉对比

特征	PCA (主成分分析)	Auto-Encoder (自编码器)
背景纯净度	较低。重构图像的背景通常带有明显的“伪影”或“底噪”（灰色斑点）。这是因为 PCA 是线性变换，舍弃的低方差分量中包含部分背景信息，重构时无法完美回归到纯黑（0 值）	极高。背景非常干净，几乎为纯黑。这是因为 Decoder 输出层使用了 Sigmoid 激活函数，将像素值强制映射在[0,1]之间，且网络学会了将背景区域的权重推向 0
边缘清晰度	相对锐利，但有震荡。数字的边缘保留了较硬的线条感，但伴随着吉布斯现象般的边缘震荡	相对平滑，略带模糊。数字边缘看起来比较柔和，甚至有些模糊。这是因为损失函数 MSE 倾向于对不确定的边缘像素取平均值
结构还原能力	一般。在 30 维的低维空间下，PCA 可能会丢失部分结构信息，导致相似数字（如 3/5，4/9）特征混淆	较强。经过充分训练的 AE 能够利用非线性能力，更好地捕捉数字的拓扑结构（如圆圈、缺口），即使在模糊的情况下也能正确区分数字类别

表 2 两种模型（PCA 和 Auto-Encoder）特性总结

维度	PCA (主成分分析)	Auto-Encoder (自编码器)
训练成本	低。计算协方差矩阵和特征值，速度极快，无需 GPU	高。需要迭代训练数千次，依赖算力和超参数调整（学习率、网络结构）
可解释性	强。特征向量（Eigen-digits）具有明确的数学和视觉含义（代表主要的笔画模式）	弱。隐空间的数值和网络权重通常是“黑盒”，难以直接解释
泛化能力	对训练集以外的数据（如非数字图像）也有一定重构能力（虽效果差）	容易过拟合于训练数据分布，对分布外数据（OOD）重构效果极差

3.3 自编码器（Auto-Encoder）模型改进点

3.3.1 移除 Dropout 层——针对重建任务的优化

在[12]的 Encoder 中使用了 nn.Dropout()。Dropout 的作用是随机“丢弃”一部分神经元以防止过拟合。但在自编码器的重建任务中，特别是当模型还不够大、训练时间不够长时，Dropout 实际上是在给图像“加噪声”。这会导致模型为了“稳健”而输出一种平均化的、模糊的结果。所以应该去掉！

3.3.2 改进 Encoder 最后一层的激活函数

在[12]中，Encoder 的最后一层是 nn.ReLU()。这意味着压缩后的 30 维向量（Latent Code）全部是非负数（小于 0 的都被置为 0 了）。这极大地限制了隐空间（Latent Space）的表达能力，丢弃了一半的信息维度。所以应该去掉 Encoder 最后一层的 ReLU，让隐变量可以是负数，包含更多信息！

代码段 2: CNN 图像识别模型 (去除 Dropout 和 ReLU 版本)

```
self.encoder = nn.SequentialCell([
    nn.Dense(in_channel, 512),
    nn.Dense(512, 128),
    nn.ReLU(),
    # nn.Dropout(),    <-- 去掉 Dropout
    nn.Dense(128, embeded_channel),
    # nn.ReLU()        <-- 去掉 Encoder 最后一层的激活函数, 让隐空间更自由
])
```

3.3.3 增加隐变量维度 (Embed Size)

当前 `cfg.embed_size = 30`。将 1024 维 (32x32) 压缩到 30 维, 压缩率极高 (约 34 倍压缩)。对于全连接网络来说, 30 个数字可能不足以完美记住所有笔画细节。故将 `cfg.embed_size` 调整为 64。

代码段 3: 增加隐变量维度

```
cfg = edict({..., 'embed_size': 64, ...})
```

3.3.4 增加训练过程可视化 (Loss 曲线)

在 MindSpore 中, `Model.train` 默认的 `LossMonitor` 只能在控制台打印日志, 无法直接返回数据用于绘图。为了绘制训练曲线, 我们需要自定义一个 `Callback` 类来记录每个 Step 或 Epoch 的 Loss 值。

代码段 4: 自定义 Callback 类

```
class LossHistory(ms.train.callback.Callback):
    def __init__(self):
        super(LossHistory, self).__init__()
        self.losses = []                # 记录每个 step 的 loss
        self.epoch_losses = []          # 记录每个 epoch 的平均 loss

    def step_end(self, run_context):
        cb_params = run_context.original_args()  # 获取当前 step 的上下文信息
        loss = cb_params.net_outputs             # 获取网络输出的 loss 值

        # 处理 loss, 如果是 Tensor 则转为 numpy
        if isinstance(loss, (tuple, list)):
            if isinstance(loss[0], ms.Tensor) and isinstance(loss[0].asnumpy(), np.ndarray):
                loss = loss[0]
        if isinstance(loss, ms.Tensor):
            loss = float(loss.asnumpy())

        self.losses.append(loss)
```

对于 Auto-Encoder, 这是一个 **重构任务** (Reconstruction Task), 而非分类任务 (Classification Task)。因此, 通常衡量标准是 **Loss (MSE, 均方误差)**, 代表重构图像与原图的差异。“准确率 (Accuracy)” 概念在这里不适用。如果 Loss 下降, 说明重构得越像, 这就代表“效果越好”。

代码段 5：修改后的训练代码

```
# .....（以上不做改动）.....
# 实例化自定义的记录器
history_cb = LossHistory()

loss_cb = LossMonitor(per_print_times=100)
config_ck = CheckpointConfig(save_checkpoint_steps=100, keep_checkpoint_max=1)
ckptpoint_cb = ModelCheckpoint(prefix='auto_encoder', directory='./exp/', config=config_ck)

print("===== Starting Training =====")
# 将 history_cb 加入回调列表
model.train(20, train_ds, callbacks=[loss_cb, ckptpoint_cb, history_cb], dataset_sink_mode=False)
print("===== finish Training =====")

# .....（使用 matplotlib 绘制 Loss 曲线，略）.....

# 分析打印
print(f'初始 Loss: {history_cb.losses[0]:.4f}')
print(f'最终 Loss: {history_cb.losses[-1]:.4f}')
```

3.4 自编码器（Auto-Encoder）改进模型运行结果

曲线**整体单调下降并趋于收敛**，没有出现发散（突然飙升）或过早停止下降的情况。表明网络结构设计合理，学习率（0.001）得当，优化器（Adam）工作正常。这次训练时长增加了 4 倍，模型有了**充足的时间**去学习细节。

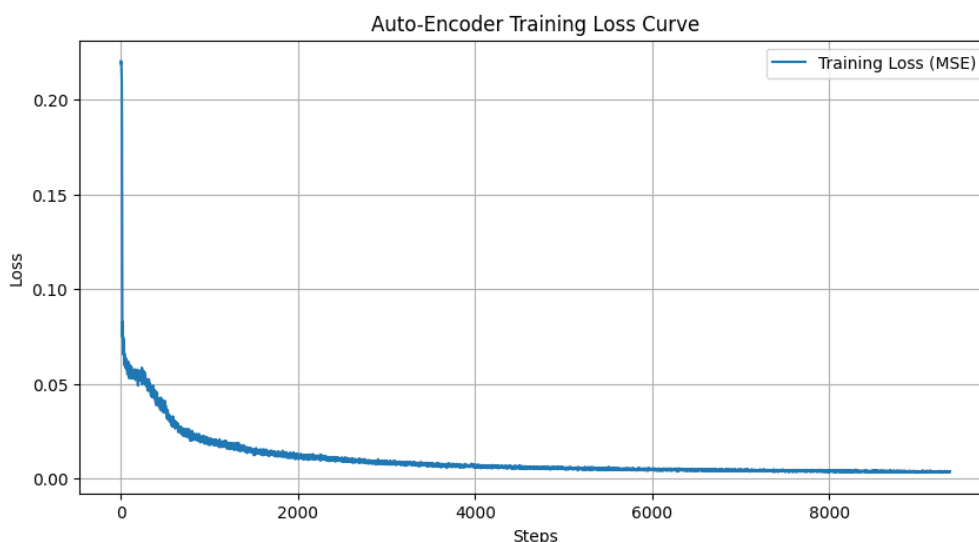


图 4 自编码器（Auto-Encoder）改进模型训练 Loss 曲线

初始 Loss: 0.2194
最终 Loss: 0.0036

图 5 自编码器（Auto-Encoder）改进模型训练初始/最终 Loss 值

观察验证结果，发现改进模型修复了先前 3.2 节提到的欠拟合的缺陷，模型对数字的分辨能力得到大幅提升！

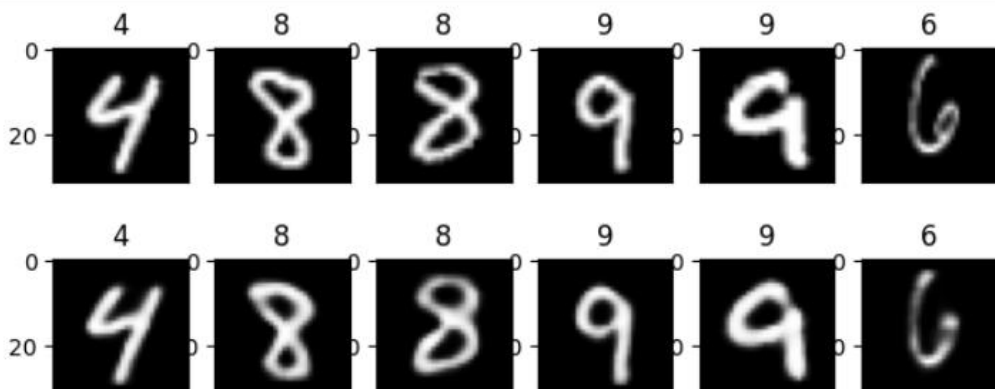


图 6 自编码器（Auto-Encoder）改进模型在验证集上的结果

四、实验结论与讨论

4.1 实验结论

本实验对比了线性降维方法 PCA 与非线性神经网络模型 Auto-Encoder 在图像压缩任务上的表现。实验结果表明：

PCA 作为稳健的线性基准，计算高效且能保留主要轮廓，但无法消除背景噪声；Auto-Encoder 在经过参数调优（解决欠拟合）后，凭借其非线性映射能力，实现了背景更纯净、语义结构更准确的图像重构。

虽然 MSE 损失函数导致了轻微的边缘模糊，但优化后的 Auto-Encoder 在去噪能力和复杂特征还原上显著优于 PCA，证明了深度学习在处理非线性数据流形时的优势。

4.2 改进空间

针对 MSE 损失函数导致的边缘模糊问题，可尝试引入感知损失（Perceptual Loss）或对抗损失（GAN）以提升图像锐度与细节还原度；

针对全连接网络破坏图像空间结构的问题，应升级为卷积自编码器（Convolutional Auto-Encoder）以更好地提取局部特征；

此外，还可探索变分自编码器（VAE）或进一步优化隐变量维度与学习率调度策略，从而在保持高压缩率的同时实现更高质量的图像重构。